Uppsala University



Machine Learning

Assignment 6

Contents

1 Oral exam question contribution 5

2 The k-means Algorithm 6

3 Probabilistic PCA 9

4 * The EM-algorithm for soft clustering: Univariate Two-Component Normal Mixture 11

General information

- The recommended tool in this course is R (with the IDE R-Studio). You can download R **here** and R-Studio **here**. You can use Python and Jupyter Notebooks, although the assignments may use data available only through the R package, a problem you would need to solve yourself.
- Report all results in a single, `*.pdf`-file. *Other formats, such as Word, Rmd, Jupyter Notebook, or similar, will automatically be failed.* Although, you are allowed first to knit your document to Word or HTML and then print the assignment as a PDF from Word or HTML if you find it difficult to get TeX to work.
- The report should be written in English.
- If a question is unclear in the assignment. Write down how you interpret the question and formulate your answer.
- You should submit the report to **Studium**. Deadlines for all assignments are **Sunday 23.59**. See **Studium** for dates. Assignments will be graded within 10 working days from the assignment deadline.
- To pass the assignments, *you should answer all questions not marked with **, and get at least 75% correct.
- To get VG on the assignment, *also the questions marked with a ** should be answered. Sometimes you can choose between different VG assignments, then this is explicitly stated. To get VG you need 75% both on questions to pass and the VG part of the assignment. VG will only be awarded on the first deadline of the assignment.
- A report that does not contain the general information (see the **template**), will be automatically rejected.
- Every assignment report must include one oral exam question of your own together with a short model answer, following the oral exam question contribution instructions in this assignment. You are not allowed to use large language models, such as ChatGPT, at all for this contribution. State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.
- When working with R, we recommend writing the reports using R markdown and the provided **R markdown template**. The template includes the formatting instructions and how to include code and figures.
- Instead of R markdown, you can use other software to make the pdf report, but you should use the same instructions for formatting. These instructions are also available as **a PDF produced from the R markdown template**.
- The course has its own R package `uuml` with data and functionality to simplify coding. To install the package just run the following:

  1. `install.packages("remotes")`
  2. ```
     remotes::install_github("MansMeg/IntroML",
     subdir = "rpackage")
     ```
     ```

- Many of the exercises can be checked automatically using the R package `markmyassignment`. Information on how to install and use the package can be found **here**. *Don't include* ***`markmyassignment`*** *results in the report.*
- We collect common questions regarding installation and technical problems in a course Frequently Asked Questions (FAQ). This can be found **here**.
- You may use large language models (LLMs), such as ChatGPT, like you may discuss with fellow students: to learn, ask for explanations, and get feedback. You may not use LLMs to produce the solution you submit. You must be able to explain every part of your own assignment at the oral exam, including your code, results, and reasoning, even when it is not covered by the published oral exam questions. If course staff have concerns about your understanding or the authorship of any part of your submission, you may be asked to explain that part at the oral exam.
- For optional LLM study support, first upload or paste the `AI_TUTOR.md` instructions into a new conversation and ask the tool to follow them. You may then upload the full teacher-provided assignment instructions, including supplied example code. Ask for explanations, incremental hints, and separate practice examples, not completed solutions. You remain responsible for following the course rules even if the tool offers a solution.
- Do not upload your own report, answers, or assessed code to an LLM. Do not share your solutions (text or code) with other students, but feel free to discuss concepts and approaches. Sharing your solutions may also be considered cheating. During scheduled support sessions held on Zoom, only share your screen in a separate room with a teaching assistant.
- Where an exercise explicitly asks you to run an LLM and report its output, you may do so for that experiment. Your explanations and analysis must be your own. This permission does not apply to the oral exam question contribution, for which LLM use is prohibited entirely.
- Scheduled support sessions are for questions about the assignments and course material. Do not hesitate to ask! Ask questions early if instructions are unclear or you encounter possible bugs, so you do not lose time unnecessarily.
- If you have any suggestions or improvements to the course material, please post in the course chat feedback channel, create an issue **here**, or submit a pull request to the public repository.

Automatic feedback with `markmyassignment`

The R package `markmyassignment` (MMA) provides automatic feedback on the functions listed in the selected check configuration. Define or load your own functions in R before running the checks. Use the `tasks` argument to check one function by name; omitting it checks every function in the selected configuration. Functions that depend on other functions need those definitions loaded too.

To select the checks for this assignment, run the following code in R:

```
install.packages("markmyassignment") # Install once
library(markmyassignment)
assignment_path <-
  paste("https://github.com/MansMeg/IntroML/",
        "blob/master/assignments/tests/assignment6/assignment6.yml", sep="")
set_assignment(assignment_path)
# List the function names available in this configuration
show_tasks()
# To check one function, replace function_name with its name:
# mark_my_assignment(tasks = "function_name")
# To check your code/functions, just run
mark_my_assignment()
```

The checks provide feedback on selected tasks. They do not assess the whole assignment or award marks. Print your complete function definitions in the PDF report as required by the general instructions, even when the checks pass. *You should not need to include MMA results in the report.* The supplied checks run R code. Python implementations are assessed from the report and cannot be run directly through MMA.

This configuration checks the basic k-means and pPCA functions. The EM section below gives a separate configuration for the VG functions.

1 Oral exam question contribution

Write one possible oral exam question of your own for this assignment together with a short model answer. Include both in your assignment report.

The question should:

- concern material from this assignment or the associated reading or lectures,
- be suitable for a 1–3 minute oral answer, possibly using a whiteboard,
- test conceptual understanding rather than only calculation or code,
- not already be included in the oral exam question list and not be very close to an existing question (e.g. just different numbers).

Note! You are not allowed to use large language models, such as ChatGPT, at all for this oral exam question contribution.

State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.

Introduction to Unsupervised Learning

Background reading: Follow the general reading instructions for Block 6.

Implementation references in ESL. For the basic k-means tasks, consult Algorithm 14.1 and Equation 14.31 in Hastie et al. (2009). For the optional EM VG section, also consult Section 8.5.1, including Algorithm 8.1. These specific excerpts supplement the conceptual background in the Block 6 reading.

This computer assignment will look into two to three widespread algorithms for unsupervised learning.

The dataset `iris` contain data on petal and sepal length and width for three different species of iris flowers. In addition, we will also use the `faithful` data on the Old Faithful Geyser in Yellowstone. The data consists of the eruption time (in minutes) and the waiting time (in minutes) until the next eruption. Finally we will also use the `mixture_data` from Table 8.1 in Hastie et al. (2009).

To access the data, just use:

```
data("iris")
data("faithful")
library(uuml)
data("mixture_data")
```

For details, run `?iris`, `?faithful`, or `?mixture_data` in R.

2 The k-means Algorithm

We are now going to build a k-means algorithm from scratch.

Data convention: Keep cluster labels in `C`, separate from the measurement columns in `X`. For the iris experiment, `X` is 150×2, `C` has length 150, and `m` is 3×2.

1. (1p) Visualize `iris` as a scatterplot of `Sepal.Width` and `Petal.Width`. Show the three species (the `Species` variable) using different colors. *Hint!* Use `ggplot2` and geom_point.

2. (1p) Manually allocate each observation in `faithful` to one of two clusters, using the `eruptions` and/or `waiting` variables.

3. (1p) Make a scatterplot of `eruptions` and `waiting`, colouring observations by your manual clusters. One figure covers Questions 2.2 and 2.3.

4. (2p) **Implementation (1p).** Implement `compute_cluster_means(X, C)` following Step 1 of Algorithm 14.1 in Hastie et al. (2009). The inputs are an $n \times p$ measurement matrix `X` and a length-n vector `C` of cluster labels $1, \ldots, K$. Return a $K \times p$ matrix `m`, with row k containing the means for cluster k. Do not include a cluster-ID column.

   Example (1p). Include the inputs, a call and actual cluster means from your implementation, with a brief explanation. Reuse the supplied example below.

```
set.seed(4711)
X <- as.matrix(faithful)
C <- sample(1:3, nrow(X), replace = TRUE)
m <- compute_cluster_means(X, C)
m

##      eruptions  waiting
## [1,]  3.416354 70.57576
## [2,]  3.571106 71.44706
## [3,]  3.487659 70.72727
```

5. (2p) **Implementation (1p).** Implement the second step in Algorithm 14.1 of Hastie et al. (2009) and call the function `compute_cluster_encoding(X, m)`. The function should take an $n \times p$ measurement matrix `X` and a $K \times p$ matrix `m` with one cluster mean per row. Return a length-n vector of cluster labels $1, \ldots, K$.

   Example (1p). Include the inputs, a call and actual labels from your implementation, with a brief explanation. Reuse the supplied example below.

```
C <- compute_cluster_encoding(X, m)
C[1:10]
## [1] 2 1 2 1 2 1 2 2 1 2
```

6. (2p) **Implementation (1p).** Combine these functions into `k_means(X,K)`, taking an $n \times p$ measurement matrix `X` and the number of clusters K. Randomly initialise the cluster labels and iterate until no assignments change. Return a list with `C` (length n) and `m` ($K \times p$). If any cluster is empty, either initially or after an update, restart the run with new random cluster assignments.

   Example (1p). Include the inputs, a call and compact actual `C` and `m` output, with a brief explanation. Reuse one of the ten runs below and report its seed.

7. (2p) **Implementation (1p).** Implement `k_means_W(X,C)` that computes the cluster-size-weighted within-point scatter (WPS) of Eq. 14.31 in Hastie et al. (2009):

$$\mathrm{WPS}(C) = \sum_{k=1}^{K} n_k \,\mathrm{SSE}_k, \qquad \mathrm{SSE}_k = \sum_{i:C_i=k} \|x_i - \bar{x}_k\|^2,$$

where n_k is the number of observations in cluster k and $\bar{x}_k$ is its mean vector.

Fitting loss and comparison statistic. The usual k-means fitting loss is $\mathrm{SSE}(C) = \sum_k \mathrm{SSE}_k$. Keep the usual mean and nearest-mean updates in `k_means()`; use the weighted WPS above for `k_means_W()` and the run comparisons below. Because of the extra n_k factors, WPS can rank clusterings differently from SSE and need not decrease at every k-means update.

Example (1p). Include the inputs, a call and actual WPS from your implementation, with a brief explanation. Reuse the supplied example or a run in the experiment below.

```
set.seed(4711)
X <- as.matrix(faithful)
C <- sample(1:3, nrow(X), replace = TRUE)
k_means_W(X, C)
## [1] 4601439
```

8. (1p) Run your `k_means(X,K)` ten times per dataset with new random initialisations: `faithful` with $K = 2$, and `iris` (`Sepal.Width`, `Petal.Width`) with $K = 3$. Present each run's final WPS in a table for each dataset.

   Hint! Set `X` and `K` for each dataset. Set the seed once before the loop for reproducible results with different initialisations.

```
set.seed(4711)
results <- vector("list", 10)
wps <- numeric(10)
for (r in seq_len(10)) {
  results[[r]] <- k_means(X, K)
  wps[r] <- k_means_W(X, results[[r]]$C)
}
```

9. (1p) For each dataset, plot the runs with the smallest and largest WPS among your ten runs. Choose any tied runs. If the selected clusterings are identical, one plot is sufficient; state this. Compare WPS only within each dataset.

10. (2p) How much did the final WPS vary between initialisations? What does this suggest about sensitivity to initialisation for each dataset?

    Note! Small or zero differences are valid results. You are not expected to obtain a particular gap between the smallest and largest WPS.

3 Probabilistic PCA

Probabilistic PCA (pPCA) describes noisy observations using fewer latent coordinates:

$$\mathbf{x} = \mathbf{b} + \mathbf{W}\mathbf{h} + \boldsymbol{\varepsilon}, \qquad \mathbf{h} \sim N(0, I_K), \quad \boldsymbol{\varepsilon} \sim N(0, \sigma^2 I_D),$$

where the latent coordinates and noise are independent, and I_K and I_D are identity matrices. For example, five sensor readings might reflect two shared underlying factors plus measurement noise. pPCA gives a lower-dimensional description of these correlated measurements; you will simulate this structure in the final task.

The model description is adapted from Goodfellow et al. (2016, Section 13.1). The model and tasks are specified here, so no reading of that section is required. For background on pPCA, use the general reading instructions for Block 6.

This also prepares you for the variational autoencoders (VAEs) in Assignment 7. In pPCA, $\mathbf{b} + \mathbf{W}\mathbf{h}$ acts as a linear decoder, mapping latent coordinates to observation means. A VAE uses a neural network decoder to model nonlinear relationships and learns an encoder that approximates the distribution of latent coordinates given an observation.

Example: Generating a pPCA observation *No submission required*

With $\mathbf{W} = (-1, 3)^T$, $\mathbf{b} = (0.5, 2)^T$, $h = 1$, and no noise, $\mathbf{x} = (-0.5, 5)^T$. Changing h moves the point along the line through $\mathbf{b}$ in direction $\mathbf{W}$. Adding noise scatters observations around this line.

1. (2p) **Implementation (1p).** Implement `pPCA(W, b, sigma2)` to return one simulated observation as a vector of length D. Here `W` is $D \times K$, `b` has length D, and `sigma2` is the noise variance. *Hint!* Use `sqrt(sigma2)` as the noise standard deviation.

   Example (1p). Include the parameter inputs, a call, seed and an actual simulated observation from your implementation, with a brief explanation. Reuse an observation from one of the simulations below.

2. (1p) Simulate 300 observations with $\mathbf{W} = (-1, 3)^T$, $\mathbf{b} = (0.5, 2)^T$, and $\sigma^2 = 1$. Run `set.seed(4711)` before simulating. Store this baseline as `X_base`, a 300×2 matrix with one observation per row, and plot it as in Figure 1.

3. (1p) Describe the different parts of the model. What are the parameters? What are the latent variables?

4. (1p) *Before simulating, predict* the effect of each of these separate changes to the baseline: $\mathbf{b} = (2.5, 2)^T$; $\mathbf{W} = (3, 1)^T$; and $\sigma^2 = 4$. For each change, reset the seed to 4711 and simulate 300 observations. Show the baseline and three settings in one figure with common axis limits; this also covers Question 3.2. Afterwards, briefly explain whether the results match your predictions.

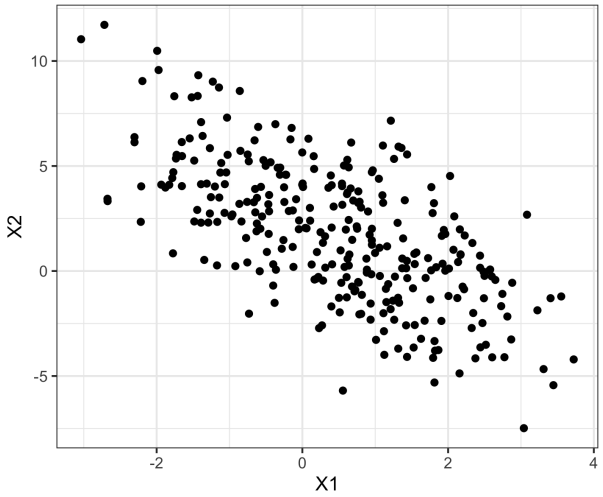


Figure 1: Data from a pPCA model with $\mathbf{W} = (-1, 3)^T$, $\mathbf{b} = (0.5, 2)^T$ and $\sigma^2 = 1$

5. (1p) Run PCA on the baseline data. Estimate the mean and the loading vector, accounting for the known baseline noise variance:

```
sigma2 <- 1
pr <- prcomp(X_base, center = TRUE, scale. = FALSE)
b_hat <- pr$center
W_hat <- sqrt(max(pr$sdev[1]^2 - sigma2, 0)) * pr$rotation[, 1]
```

   Why subtract the noise variance? For one latent coordinate, $\mathrm{Cov}(\mathbf{x}) = \mathbf{W}\mathbf{W}^T + \sigma^2 I$, so the leading eigenvalue is $\|\mathbf{W}\|^2 + \sigma^2$. Centering removes the translation by $\mathbf{b}$; changing only $\mathbf{b}$ leaves centered PCA unchanged.

6. (1p) Compare `b_hat` and `W_hat` with the generating parameters. Explain how the results connect pPCA to classical PCA. Allow for sampling variation and an arbitrary sign of the principal direction.

7. (2p) Finally, simulate 300 observations from a probabilistic PCA model with five observed variables and two latent coordinates per observation. Store the data in a 300×5 matrix `X`, with observations in rows. You are free to choose the parameter values $\mathbf{W}$, $\mathbf{b}$ and σ^2 yourself, but the resulting distribution should have at least one negative and one positive correlation between the Xs. Visualize the variables using a pair-wise scatterplot using `pairs(X)` in R.

4 * The EM-algorithm for soft clustering: Univariate Two-Component Normal Mixture

This task is only necessary to get one point toward a *pass with distinction* (VG) grade. Hence, if you do not want to get a *pass with distinction* (VG) point, you can ignore this part of the assignment.

Select the VG checks before checking the six mixture-model functions:

```
set_assignment(paste("https://github.com/MansMeg/IntroML/",
                     "blob/master/assignments/tests/assignment6/",
                     "assignment6-vg.yml", sep = ""))
```

To return to the basic checks, run `set_assignment(assignment_path)` again.

In the next step, we will implement the EM algorithm conceptually similar to k-means clustering. We are going to implement the EM by testing at the mixture data of Hastie et al. (2009). This dataset has been added to the course R package.

```
library(uuml)
data("mixture_data")
theta0 <- list(mu_1 = 4.12, mu_2 = 0.94, sigma_1 = 2, sigma_2 = 2, pi = 0.5)
```

Conventions: Throughout this section, `theta` contains `mu_1`, `mu_2`, `sigma_1`, `sigma_2`, and `pi`. The sigmas are standard deviations. `pi` is the mixing weight for component 2, and `gamma[i]` is the posterior probability that observation i belongs to component 2, given its value and the current parameters.

1. (2p) **Implementation (1p).** Implement `r_uni_two_comp(n, theta)`, returning a numeric vector of length `n`. For each observation, draw $\delta \sim \text{Bernoulli}(\pi)$. Draw from component 2 when $\delta = 1$, and component 1 otherwise, using the means and standard deviations in `theta`. See Eq. 8.36 in Hastie et al. (2009).

   Example (1p). Include the inputs, a call, seed and a compact extract of actual simulated observations, with a brief explanation. Reuse the 200 observations in the next task.

2. (1p) Simulate 200 observations using $\mu_1 = -2$, $\mu_2 = 1.5$, $\sigma_1 = 2$, $\sigma_2 = 1$ and $\pi = 0.3$. Visualize observations drawn in a histogram.

3. (2p) **Implementation (1p).** Implement `d_uni_two_comp(x, theta)` that computes the density value for a given set of parameters `theta` for values of `x`. *Hint!* The function `dnorm` might be handy.

   Example (1p). Include the inputs, a call and actual density values from your implementation, with a brief explanation. Reuse values from one of the density overlays below.

4. (1p) Add the theoretical mixture density to the histogram from Question 4.2 over the interval -6 to 6 (by steps of 0.01). Use a density-scaled histogram (`freq = FALSE` in `hist`). One combined figure is sufficient for Questions 4.2 and 4.4.

5. (1p) Visualize the `eruptions` variable in the `faithful` dataset using a histogram.

6. Fit a two-component mixture manually to the `eruptions` variable.

   (a) (1p) Choose μ_1, μ_2, σ_1, σ_2, and π, and overlay the resulting density on a density-scaled histogram from Question 4.5. One combined figure is sufficient for Questions 4.5 and 4.6(a).

   (b) (1p) Briefly explain how your chosen means, standard deviations, and mixing weight reflect the locations, widths, and relative sizes of the two groups in the data.

7. (2p) **Implementation (1p).** Implement `e_uni_two_comp(X, theta)`. Here `X` contains n univariate observations in an $n \times 1$ matrix. Return a vector `gamma` of length n, with one responsibility for component 2 per observation. This function should implement the expectation step in Algorithm 8.1 in Hastie et al. (2009). For definition of ϕ and θ, see Hastie et al. (2009, , Section 8.5.1). Subscript in Algorithm 8.1, e.g. in θ_2, indicates the component/cluster.

   Example (1p). Include the inputs, a call and actual responsibilities from your implementation, with a brief explanation. Reuse the supplied example below.

```
gamma <- e_uni_two_comp(mixture_data, theta0)
head(gamma)
## [1]  0.9106339 0.8716861 0.7797225 0.6645640 0.6484311 0.5178799
```

   Numerical hint: You can use `uuml::log_sum_exp()` (printed in Question 4.10): compute the weighted component log-densities, normalize in log space, then exponentiate. This avoids `0/0` when both densities underflow to zero.

   Note! Gamma values for component 1 is just 1 - `gamma`.

8. (1p) An observation has $\gamma_i = 0.6$. What weights does it receive for the two components in the M-step? Explain how this differs from a hard cluster assignment.

9. (2p) **Implementation (1p).** Implement `max_uni_two_comp(X, gamma)` that returns a list with parameters `mu_1`, `mu_2`, `sigma_1`, `sigma_2` and `pi`. This function should implement the maximization step in algorithm 8.1 in Hastie et al. (2009). Return `sigma_1` and `sigma_2` as standard deviations: take the square root of each weighted variance estimate.

   Example (1p). Include the inputs, a call and actual parameter estimates from your implementation, with a brief explanation. Reuse the supplied example below.

```
theta <- max_uni_two_comp(mixture_data, gamma)
theta
## $mu_1
## [1] 3.842941
##
## $mu_2
## [1] 1.450413
##
## $sigma_1
## [1] 1.700666
##
## $sigma_2
## [1] 1.47168
##
## $pi
## [1] 0.4883709
##
```

10. (2p) **Implementation (1p).** Implement `ll_uni_two_comp(x, theta)` to compute the model log-likelihood. *Hint!* See Eq. 8.39 in Hastie et al. (2009).

    Example (1p). Include the inputs, a call and actual log-likelihood from your implementation, with a brief explanation. Reuse the supplied call below the numerical helper.

    Numerical hint: Very small densities can underflow to zero. Use `dnorm(..., log = TRUE)` and add the log mixing weights before combining the components with the helper below. For finite inputs a and b, it uses the identity

    $$\text{log_sum_exp}(a, b) = \log\big(\exp(a) + \exp(b)\big) = m + \log\big(\exp(a - m) + \exp(b - m)\big),$$

    where $m = \max(a, b)$. Subtracting m keeps both exponentials at most one, avoiding overflow. Apply the helper to each observation and sum these log-densities over observations. The helper is available in `uuml` version 0.5.1 or later. Reinstall the package using the general instructions if needed. Use `?log_sum_exp` to read its documentation. The package function is printed below so you can inspect how it works; you do not need to define it yourself.

```
Error:
!  'log_sum_exp' is not an exported object from 'namespace:uuml'
```

```
ll_uni_two_comp(mixture_data, theta0)
## -43.1055
```

11. (2p) **Implementation (1p).** Combine the functions into `em_uni_two_comp(X, theta_0, iter)`. It takes an $n \times 1$ data matrix `X` and initial parameters `theta_0`. Run `iter` iterations, printing the current log-likelihood. Return a list with the final parameters as `theta` and an `iter` $\times$ 5 matrix `history`. Row i contains the parameters after M-step i; use columns `mu_1`, `mu_2`, `sigma_1`, `sigma_2`, and `pi`.

    Example (1p). Include the inputs, a call, likelihood prints, actual returned `theta` and a compact `history` extract, with a brief explanation. Reuse one of the runs below.

```
em3 <- em_uni_two_comp(mixture_data, theta0, iter = 3)
## Log Lik:  -41.53247
## Log Lik:  -41.11211
## Log Lik:  -40.48348
```

12. (1p) Test your algorithm on the `mixture_data` for 20 iterations. Do you get the same results for $\hat{\pi}$ as in Hastie et al. (2009, p. 275)? Note that you might get slightly different results (at the second decimal place). Also, compare your log-likelihoods with Figure 8.6 in Hastie et al. (2009). Do you have a similar result? *Hint!* There can be a slight different order depending on if you compute the log-likelihood value before or after the update of θ.

```
em20 <- em_uni_two_comp(mixture_data, theta0, iter = 20)
## Log Lik:  -41.53247
## ...
```

13. (1p) Run your EM algorithm on the `eruptions` variable of the `faithful` data and on the `Petal.Length` variable of the `iris` data until the log-likelihood has stabilised over successive iterations. For each dataset, report the estimated parameters and number of iterations used, and explain your convergence criterion. You may rerun with a larger `iter`; an automatic stopping rule is not required.

14. (1p) For each dataset in Question 4.13, show one density-scaled histogram with the fitted mixture density overlaid. Use `d_uni_two_comp(x, theta)` with the estimated parameters.

References

Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The elements of statistical learning: data mining, inference, and prediction*. Springer Science & Business Media, 2009.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.